

Review retrospective:
*the pure quartic moments, a dead tactic, and a copy-pasted
import block*

Claude Opus 4.8 (1M ctx), reviewer GPT-5.5 Pro

2026-06-05

Abstract

A soundness review of `Laplace/OneD/Quartic.lean`: the pure quartic ($L(x) = x^4/24$) Gibbs moments as the $k = 2$ specialisation of the generic even-monomial template, culminating in the degenerate affine covariance $\text{Cov}_t[ax + c, bx + d] = ab\sqrt{24/t}\Gamma(3/4)/\Gamma(1/4)$. Result: **a clean fidelity pass with two cleanups landed** — a dead, warning-producing tactic and a five-import block made redundant by the file’s own `MonomialPotential` import.

1 Setting

The pure quartic is the first genuinely degenerate posterior in the programme ($L''(0) = 0$, so Hessian-based Laplace does not apply); its moments are read off the generic k -monomial closed forms reviewed in `MonomialPotential` at $k = 2$. The headline is the covariance of affine observables, which decays as $t^{-1/2}$ rather than the nondegenerate t^{-1} .

2 What was reviewed

Thirteen declarations: the potential and its bridge `quarticPotential = kthPotential2`; the $k = 2$ specialisations of integrability, the half/full-line moments, the partition function, and the expectations; the $\langle x^2 \rangle$ and $\langle x \rangle$ specialisations; and the affine covariance.

3 Fidelity / soundness findings

Sound; no mismatch. The bridge is correct ($x^{2 \cdot 2}/(2 \cdot 2)! = x^4/24$); every $k = 2$ constant checks out (base $24/t$, prefactors $1/2$, exponents $(2n + 1)/4$ and $n/2$, Γ arguments); and the covariance reduction is faithful: expanding $\langle (ax + c)(bx + d) \rangle = ab\langle x^2 \rangle + (ad + bc)\langle x \rangle + cd$ and using $\langle x \rangle = 0$ gives $\text{Cov} = ab\langle x^2 \rangle = ab\sqrt{24/t}\Gamma(3/4)/\Gamma(1/4)$, the stated degenerate law. (As in the monomial base, the odd-moment theorems are stated for all t rather than $t > 0$ — a benign, correct generalisation, recorded not escalated.)

4 Simplifications applied

Two cleanups landed.

- **A dead, warning-producing tactic.** In `quartic_integrable_pow` the proof read `convert h using 4 <;> norm_num`, but `convert h using 4` already closes every goal — so the `norm_num` was unreachable, and Lean’s `unreachableTactic` linter was emitting a standing warning (“this tactic is never executed”). Removing `<;> norm_num` clears the warning. This is the rare proof-body edit that is unambiguously safe: the linter itself certifies the tactic dead, and (unlike the Harmonic file’s reverted block) removing it *eliminates* a warning rather than creating one.
- **A five-import block, redundant by transitivity.** The file carried the same seven-line import block as `MonomialPotential` — but it *imports* `MonomialPotential` (and `Gibbs`), so all five Mathlib imports (`Gamma.Basic`, `GaussianIntegral`, `Integral.Gamma`, `Lebesgue.Integral`, `Haar.NormedSpace`) arrive transitively. Removed; only the two project imports remain.

This is a foundation file (six consumers — the J-function quartic, the bounded-prior files, and three 2D files), so the import removal was arbitrated by a **full project build**: green at 8306 jobs (unchanged from baseline), all consumers rebuilt, declarations byte-identical, and the file now warning-free. GPT-affirmed both.

5 Disagreements and open questions

None on the mathematics.

6 Lessons

- **An `unreachableTactic` warning is a free, safe dead-code flag.** When the linter says a tactic “is never executed”, removing it is statement-preserving by construction and *improves* the build (one fewer warning) — the opposite risk profile from the `unusedVariables` case, where removing a block can *create* a warning on a frozen hypothesis. Both are caught by “rebuild with no new warnings”; here the warning count goes down.
- **Specialisation files inherit their parent’s import block by copy-paste — and usually don’t need it.** A file that `imports` the generic template it specialises gets the template’s Mathlib dependencies for free; the explicit re-imports are cruft. Worth checking on every “*X* is the $k = c$ case of *Y*” file, and the full build is the clean arbiter when the file has consumers.

7 Follow-ups

- The sibling `Sextic.lean` ($k = 3$) almost certainly carries the same copy-pasted import block and likely the same dead `<;> norm_num` pattern — a natural next review. The J-function files and the `Multi/Covariance*` trio remain.